

Lab 1 : Linear Regression with Boston housing dataset

Overview

This lab is designed to introduce beginners to the concept of linear regression, a fundamental statistical method used in machine learning to model relationships between a dependent variable and one or more independent variables. Throughout this lab, you will follow a step-by-step guide to set up your Python environment, explore a real-world dataset, and build linear regression models to predict housing prices.

Key Sections:

1. Environment Setup:

- You'll begin by creating a Python environment using Anaconda, ensuring all necessary libraries are installed and ready for use.

2. Data Presentation:

- The lab uses the Boston Housing dataset (<https://www.kaggle.com/datasets/vikrishnan/boston-house-prices>), a well-known dataset in the machine learning community. You will learn how to load and understand the structure of this dataset, which includes various features related to housing prices.

3. Data Exploration:

- Data exploration is critical before modeling. You will generate a comprehensive data profile using `ydata_profiling`, allowing you to uncover patterns, correlations, and potential issues within the data.

4. Variable Selection Criteria:

- Understanding which variables to include in your model is essential for accurate predictions. This section covers methods for selecting the most relevant variables and discusses the impact of these choices on model performance.

5. Linear Regression Implementation:

- The core of the lab focuses on building both simple and multiple linear regression models. You will learn how to split your data, train the model, and evaluate its performance using key metrics like Mean Squared Error (MSE) and R-squared.

6. Conclusion:

- Finally, you will reflect on the process and results, solidifying your understanding of linear regression and its applications.

Let's begin ...

Introduction

Linear regression is a fundamental technique in machine learning used to model the relationship between a dependent variable and one or more independent variables. In this lab, you will learn how to implement and evaluate linear regression models using Python.

Part 1: Environment Setup

Before starting with the implementation, you need to set up your environment. We will use Anaconda, a popular Python distribution that simplifies package management and deployment.

Step 1: Install Anaconda

1. Download the Anaconda installer for your operating system from the official Anaconda website: <https://www.anaconda.com/products/distribution>.
2. Follow the instructions provided on the website to install Anaconda on your machine.

Step 2: Create a New Conda Environment

1. Open the Anaconda Prompt (Windows) or Terminal (macOS/Linux).
2. Create a new environment by running the following command:

```
conda create -n linear_regression_env python=3.8
```

3. Activate the environment with the following command:

```
conda activate linear_regression_env
```

Step 3: Install Required Packages

In the activated environment, install the necessary packages using the following command:

```
pip install numpy pandas scikit-learn ydata-profiling
```

Part 2: Data Presentation

The dataset we will use in this lab is the Boston Housing dataset, which contains information about housing in Boston. The goal is to predict the median value of owner-occupied homes (in \$1000s) based on various features.

Download the dataset using the following code:

```
import pandas as pd
data = pd.read_csv("HousingData.csv")
print(data.head())
```

More details about the dataset are available in Kaggle :
<https://www.kaggle.com/datasets/vikrishnan/boston-house-prices>

Part 3: Data Exploration

Before we proceed with the regression, it's important to explore the data to understand its characteristics. We will use the `ydata\_profiling` package to generate a detailed report.

Generate the report with the following code:

```
from ydata_profiling import ProfileReport

profile = ProfileReport(data, title='Boston Housing Data Report')
profile.to_file('boston_housing_report.html')
```

Analyze the Report:

Review the profiling report to understand the data better. Pay attention any potential correlations between features.

Part 4: Variable Selection Activity

Variable selection is a crucial step in building an effective linear regression model. In this activity, you will apply different methods to select the most relevant variables from the Boston Housing dataset and assess the impact of your choices on the model's performance.

4.1 Exploring Correlations

Objective: Understand the relationships between different variables and the target variable (MEDV).

Instructions:

- Load the Boston Housing dataset into a Pandas DataFrame (as done in previous sections).
- Generate a correlation matrix to visualize the relationships between all the features and the target variable (MEDV).
- Identify the top 3 features that have the highest positive or negative correlation with MEDV.

```
%matplotlib inline

import seaborn as sns
import matplotlib.pyplot as plt

# Generate the correlation matrix
corr_matrix = data.corr()

# Display the correlation matrix
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()

# Identify top 3 correlated features with MEDV
print(corr_matrix['MEDV'].sort_values(ascending=False).head(4))
```

Discussion:

- Which features are most strongly correlated with MEDV?
- Do you think these features will be good predictors? Why or why not?

Part 5: Linear Regression on Boston Housing Dataset

Now that we have explored the data, let's build a linear regression model to predict the median value of homes (MEDV).

Step 1: Simple Linear Regression

We will start with a simple linear regression using one feature to predict MEDV. Use the following code:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Select the feature and target
X = data[['RM']]
y = data['MEDV']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create the model and train it
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f'Mean Squared Error: {mse}')  
print(f'R-squared: {r2}')
```

1. Discuss the chosen variable (RM) for prediction.

Step 2: Multiple Linear Regression with OLS

Next, we will perform multiple linear regression using multiple features to predict MEDV. Use the following code after adapting it with outcomes from section 4. Consider only the following variables :

- Dependent variable (target): **MEDV**
- Independent variables (features): **NOX, RM, DIS, RAD, TAX, PTRATIO, B**

```
# Select features and target  
X = data[['NOX', 'RM', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B']]  
y = data['MEDV']  
# Split the data into training and testing sets  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
# Create the model and train it  
model = LinearRegression()  
model.fit(X_train, y_train)  
  
# Make predictions  
y_pred = model.predict(X_test)  
  
# Evaluate the model  
mse = mean_squared_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
  
print(f'Mean Squared Error: {mse}')  
print(f'R-squared: {r2}')
```

1. Discuss the result (comment)

Step 3: Multiple Linear Regression with Gradient Descent

So far, you used `LinearRegression` from `scikit-learn`, which computes the coefficients analytically using the Normal Equation. In the following, you will implement **Gradient Descent** to estimate the regression coefficients iteratively.

A. Prepare your data

- Use `X = data[['NOX', 'RM', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B']]` and `y = data['MEDV']`.
- Standardize the features (subtract mean and divide by std) so Gradient Descent converges faster.

```
# MLR with Gradient Descent
from sklearn.preprocessing import StandardScaler
import numpy as np

# Features and target
X = data[['NOX','RM','DIS','RAD','TAX','PTRATIO','B']]
y = data['MEDV']

# Scale features for faster convergence
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Add bias column (for  $\beta_0$  intercept)
m, n = X_scaled.shape
X_b = np.c_[np.ones((m, 1)), X_scaled]
```

B. Initialize parameters

```
# Initialize coefficients ( $\beta$ )
beta = np.zeros(n + 1) #  $\beta_0, \beta_1, \dots, \beta_n$ 
```

C. Gradient Descent loop

```
learning_rate = 0.01
n_iterations = 1000

for i in range(n_iterations):
    gradients = -2/m * X_b.T.dot(y - X_b.dot(beta))
    beta = beta - learning_rate * gradients
```

D. Step 4. Results & Comparison

```
print("Estimated coefficients ( $\beta$ ):")
print(beta)

# Predictions
y_pred = X_b.dot(beta)

# Evaluation
from sklearn.metrics import mean_squared_error, r2_score
print("MSE:", mean_squared_error(y, y_pred))
print("R²:", r2_score(y, y_pred))
```

Question

- β_0 is the intercept.
- $\beta_1 \dots \beta_n$ are the slopes for NOX, RM, DIS, RAD, TAX, PTRATIO, B.
- Compare these values to `LinearRegression().fit(X, y)` coefficients.

💡 *Hint:* If results don't converge, adjust `learning_rate` (too large $\rightarrow$ divergence, too small $\rightarrow$ very slow).

Part 6 : Bonus

Q1. Handling Missing Values

In real datasets, some columns may contain missing values (NaN).

- Identify if there are missing values in your dataset.
- Explain how you handled them and justify your choice.
- Try at least two of the following strategies and compare results:
 - Drop rows or columns with missing values.
 - Replace missing values with a constant (e.g., 0).
 - Use summary statistics (e.g., mean, median, or mode).
 - Use a more advanced imputation method (e.g., regression, k-NN imputation).

💡 *Hint:* Use `data.isna().sum()` to detect missing values. Think about the **trade-offs** between losing data (dropping) and introducing bias (filling).

Q2. Collinearity Between Variables

In multiple regression, predictors may be highly correlated with each other (collinearity), which can make coefficient interpretation unstable.

- Explore whether collinearity exists between your chosen predictors.
- Explain why this is a problem in regression analysis.
- Suggest and test at least one strategy to deal with it, such as:
 - Removing or combining correlated variables.
 - Using dimensionality reduction (e.g., PCA).
 - Applying regularized regression (e.g., Ridge, Lasso).

💡 *Hints:*

- Use the **correlation matrix** (`data.corr()`) or a heatmap (`sns.heatmap(...)`) to spot highly correlated pairs.
- Focus on whether features like TAX, RAD, or DIS show strong correlations.
- Think about how model performance and interpretability are affected.

Save the Notebook and Share

1. Save the notebook by selecting File > Save As in the Jupyter Notebook interface.
2. Export the notebook to a PDF to submit your lab assignment.

Conclusion

In this lab, you learned how to set up your Python environment, explored the Boston Housing dataset, and implemented both simple and multiple linear regression models. You also evaluated these models using metrics like Mean Squared Error and R-squared. These steps provide a strong foundation for understanding and applying linear regression in real-world scenarios.

OPTIONAL

In this section, you will apply the same linear regression techniques learned with the Boston Housing dataset to a new dataset from Kaggle: Car Price Prediction Dataset.

7.1 Load and Explore the Dataset

1. Download the dataset from Kaggle and load it into a Pandas DataFrame.

```
import pandas as pd
car_data = pd.read_csv("CarPrice_Assignment.csv")
car_data.head()
```

2. Perform **data cleaning**:

- Check for missing values.
- Explore categorical variables (e.g., fuel type, car company).
- Encode categorical variables using `pd.get_dummies()` if necessary.

3. Generate a profiling report (optional):

```
from ydata_profiling import ProfileReport
profile = ProfileReport(car_data, title="Car Price Data Report")
profile.to_file("car_price_report.html")
```

7.2 Data Exploration & Variable Selection

- Compute correlations between numerical features and car price.

Questions :

1. Which features are most strongly correlated with car price?
2. Do you think categorical features (e.g., fuel type, car company) might also affect price? Why?

7.3 Simple Linear Regression

- Select one strong predictor (e.g., `horsepower`) to predict price.
- Train and test a simple linear regression model.

Questions :

- How well does horsepower alone predict car price?
- Interpret the coefficient: how does car price change with an increase of 1 unit in horsepower?

7.4 Multiple Linear Regression

- Use multiple features (e.g., `horsepower`, `engine_size`, `curbweight`, `highwaympg`) to predict car price.

Questions :

1. Compare the performance of the multiple regression model with the simple regression. Which performs better?
2. Which predictor seems to contribute the most to explaining car price?